Spring MVC Controller returns “ModelAndView” or “String”

http://stackoverflow.com/questions/7175509/which-is-better-return-modelandview-or-string-on-spring3-controller

The way of return ModelAndView

```
@RequestMapping(value = "/list", method = RequestMethod.GET)
public ModelAndView list(
    @UserAuth UserAuth user,
    ModelAndView mav) {

    if (!user.isAuthenticated()) {
        mav.setViewName("redirect:http://www.test.com/login.jsp");
        return mav;
    }

    mav.setViewName("list");
    mav.addObject("articles", listService.getLists());

    return mav;
}
```

The way of return String

```
@RequestMapping(value = "/list", method = RequestMethod.GET)
public String list(
    @UserAuth UserAuth user,
    Model model) {

    if (!user.isAuthenticated()) {
        return "redirect:http://www.test.com/login.jsp";
    }

    model.addAttribute("articles", listService.getLists());

    return "list";
}
```

Answer:

There is no better way. Both are perfectly valid. Which one you choose to use depends which one suits your application better - Spring allows you to do it either way.

Historically, the two approaches come from different versions of Spring. The `ModelAndView` approach was the primary way of returning both model and view information from a controller in pre-Spring 2.0. Now you can combine the `Model` parameter and the `String` return value, but the old approach is still valid.

If you return a String, it will assume that this is the name of the view to use. It will wrap this in a ModelAndView object with the string as the view and the existing model embedded as well.

Spring does a lot of 'magic' on the return types, but also on the parameter types. This allows you to write code in a style that is more intuitive for you, i.e. the following two examples are the same:

```
@RequestMapping(value="/")
public ModelAndView mainPage() {
   Model m = new Model();
   m.put("key", "value");
   return new ModelAndView(m,"main");
}
```
and
```
@RequestMapping(value="/")
public String mainPage(Model m) {
   m.put("key", "value");
   return "main";
}
```
This second variant is a little less verbose and easier to test separately. The model passed in will be an empty model (unless forwarded from some other controller).

I always use the approach where controller methods return `ModelAndView`. Simply because it tends to make the controller methods a little more terse. The method parameters are now strictly `input`parameters. And all `output` related data is contained in the object returned from the method.
The `ModelAndView` style seems to resonate with people who don't like updating input parameters to a method. Sticking to the belief that this would constitute a side effect, a dangerous pattern because you cannot reliably predict what the method is going to do - It could return data in the returned object, or it could have updated anything in any of the input arguments.
So some people will still continue to prefer `ModelAndView`.
The new style with `Model` as method parameter and returned string as view name. Seems to have come from a slightly different design approach. Here the model objects are considered to be sort of events or items that are passed to multiple handlers, before being returned to the view where they are rendered